MY EYES

Vision Beyond Sight

Technical Project Documentation

A voice-first, AI-powered mobile accessibility platform for blind and visually impaired users

Implementation basis: source tree reviewed in PSG-capstone (Flutter frontend and FastAPI backend).
Prepared: 6 October 2026

Document Status and Reading Guide

This report is implementation-led. Statements labelled **Implemented** are supported by the repository. **Configuration dependent** means code exists but a credential, device capability, or external provider is required. **Needs verification** identifies deployment details that cannot be proven from source alone. This distinction avoids treating planned behavior as delivered behavior.

Abstract

My Eyes - Vision Beyond Sight is a Flutter and FastAPI accessibility application that turns camera or user-selected images into spoken, user-specific assistance. It integrates document OCR, medicine and product reading, currency recognition, object and scene analysis, text-to-speech, voice command routing, biometric-first account access, scan history, emergency SOS, navigation, and AI Help. The design reduces touch dependence by confirming choices and returning understandable speech through an active companion profile, Alex or Maya.

Contents

1 Introduction and problem | 2 Proposed system | 3 Architecture and data flow | 4 User experience | 5 Modules | 6 Backend, API and data | 7 Setup and testing | 8 Quality, limitations and future scope

1. Introduction, Problem Statement and Objectives

Blind and visually impaired people frequently need a second person to read labels, distinguish a banknote, identify nearby items, or interpret a visual scene. Conventional apps often place critical controls behind small, visual, touch-heavy interfaces. Separate single-purpose tools also force users to switch applications and learn inconsistent interaction patterns.

My Eyes addresses this gap with one authenticated, voice-guided mobile application. A user can ask for a task, choose the camera or an existing image, receive a result, hear it through text-to-speech, and later retrieve module-specific history. The intended outcome is greater independence for reading printed text, packaged information, medicine labels, currency, objects, and surroundings.

Objectives

• Provide voice-led access to vision functions with clear spoken confirmation and errors.

• Keep camera capture and file selection as separate, reliable input paths.

• Protect personal results through authenticated, user-scoped APIs and history.

• Support selectable companion voices and repeatable instructions.

• Offer safety and mobility support through SOS and voice-guided navigation.

Scope and target users

Primary users are blind and visually impaired smartphone users. Sighted helpers may assist during onboarding, permission prompts, or ambiguous results. The system is an assistive tool: it must not be used as the sole source for medical, payment, or emergency-critical decisions.

2. Proposed System and Verified Feature Set

The Flutter client provides screens, accessibility semantics, camera/file entry, speech recognition, text-to-speech, and secure token storage. It calls a FastAPI service that validates authenticated requests, delegates to task services and AI/OCR engines, records applicable results, and returns schemas consumed by the client.

Feature	Status	Purpose / implementation evidence
Read Text (OCR)	Implemented	/ocr/scan, EasyOCR dependency, OCR engine/service and Read Text screen.
Medicine Reader	Implemented	/medicine/scan with medicine engine/service; OCR-led label reading. Not a diagnosis service.
Product Reader	Implemented	/product/scan with product reader engine/service for printed package information.
Currency	Implemented	/currency/scan with currency engine/service.
Object / Scene	Implemented	/objects/detect and /scene/describe; Ultralytics and yolov8n.pt are present.
Companions	Implemented	Alex/Maya profiles, change-companion screen, stored profile companion field.
History	Implemented	scan_history model, migration, history APIs and module history screens.
Biometric-first access	Implemented	device credential model, biometric service and /auth/biometric/login.
Google Sign-In	Implemented/configuration dependent	google_sign_in client dependency, setup guide and server google-auth dependency/service; Cloud configuration is required.
SOS, navigation, payment QR, AI Help	Implemented/configuration dependent	Dedicated client/backend routes. SOS delivery, Geoapify and Gemini need their respective configuration.

3. Complete System Architecture

The application is layered so that user interaction remains in Flutter while processing and persistent data remain on the FastAPI side.

Flutter UI and accessibility
Dashboard, onboarding, scanner, history, companion, SOS, navigation and payment screens
↓ voice command service, speech-to-text, TTS, camera/file picker, secure local storage
↓ HTTPS API requests with Bearer token
↓
FastAPI API layer
routers → dependencies/JWT user resolution → services → AI engines / PostgreSQL
↓
Response
typed JSON → Flutter result screen → active companion speech → user

Frontend architecture

The Flutter application uses Dart. Key source areas include **lib/screens** for user flows, **lib/services** for APIs, speech, file picking, biometric flow and storage, **lib/core** for routing/catalog/theme, and **lib/widgets** for reusable controls. Dependencies include camera, image_picker, file_picker, speech_to_text, flutter_tts, local_auth, flutter_secure_storage and google_sign_in.

Backend architecture

FastAPI starts in app/main.py, mounts uploads/audio static folders, registers routers, configures CORS, initializes SQLAlchemy metadata, and installs structured handlers for validation, HTTP, and unexpected errors. Routers are deliberately thin: a route accepts a file/body, obtains the database session and authenticated user, then calls the appropriate service.

Database architecture

PostgreSQL is the intended datastore through SQLAlchemy and Alembic. The database holds user/account state and application records including scan history, device credentials, emergency contacts/events, saved places, and AI conversations/messages. Production database name and connection value are deployment settings, not hard-coded facts.

4. End-to-End Data Flow

Startup: Splash screen → auth gate/session coordination → onboarding or dashboard. The dashboard starts voice assistance, exposes cards and repeats instructions through the eye/visibility control. The active companion affects visual theme and spoken output.

Vision module flow: User says or selects a task → module router resolves a VisionMode → app asks camera or file → camera capture or Android Storage Access Framework picker returns bytes → client validates non-empty input → multipart request carrying **file** and Bearer token → FastAPI dependency resolves current user → service invokes relevant engine → service formats the result and records it where implemented → JSON response → Flutter presents and speaks it.

User request	Backend route	Processing / result
Read this text	POST /ocr/scan	OCR extracts printed text; client speaks returned text.
Read my medicine	POST /medicine/scan	Medicine service reads supplied label content; user should verify critical details.
Read this product	POST /product/scan	Product reader cleans OCR text and returns useful package details when readable.
Identify currency	POST /currency/scan	Currency engine identifies a note when confidence/quality permits.
What is this?	POST /objects/detect	Object detector returns detected object information.
Describe surroundings	POST /scene/describe	Scene engine produces a description from the supplied image.

Input separation

File input does not fall back to a fresh camera capture. VoiceFilePicker calls the platform image picker with **withData: true**, returns the chosen file bytes and filename, and returns an empty result on cancellation or failure. The client must send those returned bytes to the selected module. Camera flow separately opens/captures, validates, and sends its captured image.

5. Voice-First Experience, Companions and Instructions

Speech recognition maps a user phrase to an action through the voice command service/router. TTS then reads instructions, confirmations, results and recovery guidance. The dashboard includes visible controls with semantics as a fallback, but important flows are designed to be navigable by speech.

Companion system

Two companion profiles are present: Alex and Maya. Selection is available in onboarding and the Change Companion screen. The chosen value is retained in the user profile (users.companion) and is used to resolve presentation and voice behavior; the application function does not change when a companion changes. A successful change should confirm the new companion immediately, then continue the same active flow.

Dashboard instructions

The dashboard has a visibility/eye icon and a Repeat Instructions action. The guide is built from the module catalog and teaches task selection, camera versus file selection, hearing results, history, companion changes, repeating instructions, accessibility behavior, and what to do after a recognition failure. It is designed to be replayed rather than remembered.

User journey examples

Journey	Flow
First-time user	Splash → account/onboarding → companion choice → instructions → dashboard → spoken welcome.
Existing user	Splash/auth gate → biometric/device session as available → dashboard → command or card.
Read text	Say ‘read this text’ → select camera/file → submit image → OCR → display + companion speaks result → read-text history.
Change Alex to Maya	Dashboard/change screen → select Maya → profile update → confirmation in Maya profile → return to dashboard.
Failure	Permission/network/recognition error → concise spoken cause → offer retry, another input source, or cancellation.

6. Module-by-Module Documentation

Read Text / OCR

Input is an image file from the camera or picker. The OCR engine processes readable printed text and the service returns a structured OCR response. The expected spoken behavior is to read the extracted result. If no readable text is found, the client should say so and prompt for better lighting, focus, or another image.

Medicine Reader

The module sends an image to POST /medicine/scan. It uses the medicine engine/service to read label information available in the image. It should communicate uncertainty and must not make dosage, diagnosis, or treatment claims beyond the returned text. Important medical details require independent verification.

Product Reader

POST /product/scan routes an image to the product reader. OCR is appropriate for printed product name, dates, ingredients, and other label fields when legible. Text cleaning/filtering is useful to suppress irrelevant fragments; extraction quality depends on the image.

Currency Detection

POST /currency/scan calls the currency service/engine. The result should be spoken clearly only when the system can identify the note sufficiently. A low-confidence or unrecognised result should request a retake in good light rather than guess.

Object Detection

POST /objects/detect invokes the object detection service. The repository contains Ultralytics and YOLO weights, supporting object-detection capability. Returned object names and confidence information can be spoken, including multiple detections where the response provides them.

Scene Description

POST /scene/describe uses the scene service/engine to turn an image into an environment description. The application should state when it cannot form a reliable description and invite another image.

Additional implemented modules

AI Help has POST/DELETE /ai-help endpoints for conversational help. Smart Navigation exposes saved places, geocoding, nearby search, directions and reverse lookup, using a server-side Geoapify key when configured. Emergency contacts and SOS events are user-scoped. Payment scanning is a Flutter feature with UPI parsing/services; payment confirmation must remain deliberate and secure.

7. History, Authentication and Security

Module history

A scan history record holds id, user_id, scan_type, optional input_image path, result, and created_at. The history UI includes a picker and a module history screen. A module-specific query uses scan_type so, for example, OCR history is not mixed with currency history. The composite index ix_scan_history_user_type_created supports newest-first per-user/per-module retrieval.

Authentication

The backend issues HS256 JWT access tokens with a 30-minute expiry after account/device authentication. Protected routes use OAuth2 Bearer token extraction and current-user resolution. Password utilities use pwdlib's recommended password hashing. New biometric-first accounts may have no password; legacy hashed-password accounts remain supported. The client uses local_auth and flutter_secure_storage. Google Sign-In code and setup documentation exist, but it requires Android/Google Cloud client configuration and server-side ID-token verification setup.

Security notes

The repository's security module currently defines a literal JWT secret; this is suitable only for local development and should be moved to a private environment variable before release. The .env.example correctly keeps database URLs, Gemini, Geoapify and SOS credentials as placeholders. Input validation, authenticated routes, user-scoped records and structured errors are implemented foundations; TLS, deployment secrets, rate limiting and restrictive production CORS need deployment verification.

Error handling and recovery

Condition	System response and user feedback	Recovery
Camera/file permission or cancellation	Do not crash; return to the flow with a spoken cancellation/failure message.	Grant permission, choose file again, or use camera.
Empty/unsupported image	Reject before or at API validation; explain image was unusable.	Select/capture a valid image.
OCR/detection uncertainty	Return no-result/low-confidence outcome rather than invent a result.	Improve lighting, focus, framing; retry.
Network/API unavailable	Client surfaces API error; FastAPI structured envelope handles validation/HTTP/unhandled failures.	Check connection and retry later.
Expired/invalid token	Protected dependency rejects request.	Re-authenticate, then repeat request.
TTS/speech failure	Keep readable on-screen result and report speech unavailability.	Retry speech or use controls.

8. API Documentation

All routes below are verified from router declarations. Most vision routes require an Authorization: Bearer token and multipart field **file**. Exact response field names are defined in app/schemas and should be used by clients rather than inferred.

Method and endpoint	Auth	Purpose / request
POST /auth/register	No	Create account; AccountSession response (201).
POST /auth/account/status	No	Check device/account status.
POST /auth/account/link	No/flow-specific	Link device/account session.
POST /auth/biometric/login	No/credential flow	Obtain AccountSession after device credential verification.
GET /profile/me	Bearer	Return current UserResponse.
PUT /profile/companion	Bearer	Persist active companion.
PUT /profile/update; /change-password	Bearer	Update profile or legacy password.
POST /ocr/scan; /medicine/scan; /product/scan; /currency/scan	Bearer	Multipart file submitted to named vision service.
POST /objects/detect; /scene/describe	Bearer	Multipart file for object or scene result.
POST /history/save; GET /history/all,/types,/module/{scan_type}	Bearer	Create/list/filter user scan records.
GET/POST/PUT/DELETE /emergency-contacts...	Bearer	Manage emergency contacts and primary contact.
GET /sos/status; POST /sos/trigger; POST/GET /sos-events	Bearer	SOS readiness, delivery trigger and user event records.
POST /tts/generate	Needs verification	Generate TTS response; check schema for exact payload.
POST /ai-help; DELETE /ai-help/{conversation_id}	Bearer	AI Help conversation operations.
GET/POST/PUT/DELETE /navigation/saved-places; GET geocode, nearby, categories, directions, reverse	Bearer	Navigation and place support.

Example vision request

```
POST /ocr/scan
Authorization: Bearer <access_token>
Content-Type: multipart/form-data
file=@label.jpg
```

Expected behavior: a validated response matching OCRResponse. Possible status categories include success, validation error (422), authentication failure (401), missing resource (404), and controlled/unexpected server error (5xx). The central error handlers use a clean success:false error envelope for error conditions.

9. Database and Migration Design

Table/model	Key verified fields / role
users	id PK; full_name; unique email; nullable password; phone_number; companion; created_at. Owns history, device credentials, contacts, SOS events, conversations and saved places.
scan_history	id PK; user_id FK users.id ON DELETE CASCADE; scan_type; input_image; result; created_at. Composite per-user/type/time index.
device_credentials	Biometric/passwordless device credential model present; exact schema in model source.
emergency_contacts, sos_events	Emergency destination and event records; migration adds readiness fields.
saved_places	Navigation persistence model.
ai_conversations, ai_messages	AI Help conversation persistence models.

ER-style view

users (1) ■■< scan_history
users (1) ■■< device_credentials
users (1) ■■< emergency_contacts ■■< sos_events
users (1) ■■< saved_places
users (1) ■■< ai_conversations ■■< ai_messages

Migrations

Alembic is configured through alembic.ini, alembic/env.py and version scripts. The scan_history migration c721dc2f90ab creates the table, its user foreign key and index. Later migrations cover saved places, emergency contacts/events, biometric devices, AI conversations/messages and SOS/history readiness. Standard workflow: set DATABASE_URL, run `alembic upgrade head`, then generate reviewed schema changes with Alembic revision tooling. The application additionally performs limited startup column compatibility checks; that is a resilience measure, not a replacement for migrations.

Environment configuration

Place .env in `PSG-capstone/app/.env`. Required/optional names evidenced by .env.example include DATABASE_URL, GOOGLE_OAUTH_WEB_CLIENT_ID, GEMINI_API_KEY, GEMINI_MODEL, GEOAPIFY_API_KEY, EMERGENCY_ALERT_WEBHOOK_URL, EMERGENCY_ALERT_WEBHOOK_TOKEN and EMERGENCY_ALERT_TIMEOUT. Never commit real values.

10. Folder Structure and Installation

Known implementation structure

`PSG-capstone/app/` main.py, routers/, services/, ai/, models/, schemas/, database/, security.py, dependencies.py, error_handlers.py

`PSG-capstone/alembic/` env.py and versions/

`PSG-capstone/frontend/lib/` main.dart, screens/, services/, core/, models/, widgets/

`PSG-capstone/frontend/test/` voice command, UPI, route geometry and long-press tests.

Development setup

Step	Command / action
Prerequisites	Install Python, PostgreSQL, Flutter/Dart, Android Studio + SDK, Git and a physical Android device/emulator.
Backend environment	Create/activate a virtual environment; install dependencies from app/requirements.txt.
Configure	Copy app/.env.example to app/.env; set DATABASE_URL and only the external keys required for the features being tested.
Migrate	From PSG-capstone, run alembic upgrade head after PostgreSQL is available.
Start backend	From PSG-capstone: py -m uvicorn app.main:app --reload
Frontend packages	From PSG-capstone/frontend: flutter pub get
Run frontend	Connect device/emulator and run flutter run. Ensure its API base URL reaches the backend from the device.

Startup behavior

The splash screen provides branding and an opportunity to coordinate session/auth routing before the dashboard appears. Production builds should avoid exposing raw startup exceptions: log safe diagnostics server-side, show concise user-facing spoken guidance, and offer retry rather than a stack trace.

11. Testing Strategy and Demo Flow

Testing should combine existing Flutter tests with backend API and device-level testing. Do not record unmeasured performance claims; record actual results in the Actual Result column during test execution.

ID	Feature / input	Expected result	Actual / status
TC-01	Valid printed label → OCR	Text response is shown/spoken; correct user history entry if saved.	To execute
TC-02	Blurry/blank image → OCR	No fabricated text; clear spoken retry guidance.	To execute
TC-03	Product/medicine/currency sample	Correct service route and a useful, confidence-aware response.	To execute
TC-04	Object/scene photo	Detection/description returned; multi-result layout remains accessible.	To execute
TC-05	Choose image file	Exactly selected bytes are processed; no camera capture occurs.	To execute
TC-06	Expired token	Protected request rejected; app requests authentication.	To execute
TC-07	History filter	Only current user + chosen scan type are listed newest first.	To execute
TC-08	Companion change	Profile persists; next spoken interaction uses selected companion.	To execute
TC-09	Offline/API timeout	Understandable spoken recovery, no app crash.	To execute
TC-10	SOS setup / trigger	Contacts/readiness and delivery fallback are clearly reported.	To execute

Presentation demo

Launch → splash/auth → explain Alex/Maya selection → dashboard voice prompt → read a label → identify a package/currency/object → describe a scene → open module history → replay instructions → switch companion → show file picker path → demonstrate a failed image and retry guidance → conclude with SOS/navigation/AI Help as configured features. Use pretested sample images, a reachable backend, and clear disclosure of any missing external keys.

12. Performance, Limitations and Future Enhancements

Performance and reliability

No benchmark numbers are claimed. Measure end-to-end time separately for image acquisition, upload, backend inference, database write, response parsing and TTS start. Test poor network conditions, oversized images, repeated scans, database outages and unavailable external integrations. Image size optimization should preserve text legibility for OCR.

Known limitations

• OCR and recognition are sensitive to blur, glare, low lighting, perspective, resolution and unusual fonts.

• Object and scene models can be uncertain, incomplete or contextually wrong; outputs require cautious wording.

• Some capabilities depend on network access and configured third-party credentials (Gemini, Geoapify, SOS delivery, Google Sign-In).

• Medicine and payment results are assistive information, not medical advice or an authorization to transact.

• Voice recognition, TTS availability and device permission behavior vary by Android device and language.

Future enhancements

• Offline/on-device OCR and selected vision models; multilingual speech and OCR.

• Expanded currency and medicine reference support with auditable confidence/verification flows.

• More robust scene understanding, wearable/smart-glasses integration and personalized voice prompts.

• Accessible location assistance, emergency workflows, privacy hardening, monitoring and production rate limiting.

• Formal usability testing with visually impaired participants and regression test automation.

Technology stack

Technology	Verified use
Flutter / Dart	Android mobile client, screens, services and accessibility UI.
Python / FastAPI	Backend API, routes, schemas, dependencies and error handling.
PostgreSQL / SQLAlchemy / Alembic	Persistent relational data, ORM and migrations.
JWT / pwdlib / local_auth	Session token, password hashing and client biometric access.
EasyOCR / OpenCV / Ultralytics	OCR and computer-vision processing dependencies; YOLO weights present.
flutter_tts / speech_to_text	Spoken output and voice command input.
Google services / Geoapify	Configuration-dependent identity, AI Help and navigation integrations.

Conclusion

My Eyes combines mobile accessibility design, computer vision, OCR, voice interaction, secure user data and a service-oriented backend into one assistive platform. Its core strength is the connection between input choice, authenticated processing, understandable results and repeatable speech guidance. Continued testing with real users, careful confidence handling and secure production deployment are essential to make that promise dependable in everyday use.

References

Repository source reviewed: PSG-capstone/app, PSG-capstone/alembic and PSG-capstone/frontend (local project source). Technology configuration references are documented in app/.env.example, app/requirements.txt, frontend/pubspec.yaml and frontend setup documents. External service capabilities must be verified against their current official documentation before deployment.